

Assignment — Matrix–Vector Multiplication and Determinant Approximation (RV32IMF)

Course: Computer Architecture (ELL305)

Target ISA: RV32IMF (RISC-V 32-bit Integer + Multiply/Divide + Float)

Expected Effort: ≤ 15 hours ; submission time + date -

1. Learning Objectives

By completing this assignment, you will learn to:

- Implement complex matrix operations in RISC-V assembly (RV32IMF).
- Handle row-major memory addressing and nested loops.
- Use floating-point instructions (fadd.s, fmul.s, fdiv.s) with integer indexing.
- Evaluate instruction count, register allocation, performance and optimize all.

2. Problem Statement

Write a RISC-V assembly program that performs:

1. Matrix–Vector Multiplication: $Y_i = \sum A_{ij} \times X_j$ for a 6×6 floating-point matrix A and 6×1 vector X.
2. Determinant Computation $\det(A)$ by triangularization (regular credit) or eigenvalue / singular value computation (extra credit)
3. Comparison Check: Compare $\sum Y$ and $\det(A)$; set flag = 1 if $\sum Y < \det(A)$, else 0.

3. Constraints

- ISA subset: RV32IMF (integer, multiply/divide, and single-precision float).
- Use RV32F instructions: flw, fsw, fadd.s, fmul.s, fdiv.s, fmv.s.
- Integer registers (x5–x15) for loop counters and addresses.
- Implement subroutines: matvec_mul, det_approx, compare_sum_det.
- Follow RISC-V calling conventions for arguments and returns.

4. Template Data Section

.data

N: .word 6

A: .float 1.0, 2.0, 3.0, 4.0, 5.0, 6.0,
2.0, 1.0, 2.0, 1.0, 2.0, 1.0,
3.0, 0.0, 1.0, 2.0, 1.0, 0.0,
1.0, 2.0, 3.0, 4.0, 3.0, 2.0,
2.0, 1.0, 0.0, 1.0, 2.0, 1.0,
1.0, 3.0, 2.0, 4.0, 1.0, 5.0

X: .float 1.0, 2.0, 3.0, 4.0, 5.0, 6.0

Y: .space 24

det_result: .float 0.0
comparison_flag: .word 0

5. Functional Requirements

Functions:

matvec_mul – Computes $Y = A \times X$ using nested loops.

det_comp – Computes determinant.

compare_sum_det – Compares $\sum Y$ with $\det(A)$ using flt.s.

main – Calls subroutines and stores results to memory.

6. Deliverables

1. Assembly File: matvec_det_rv32imf.S
2. Execution Proof: Screen video capture showing input, execution and output in IDE. 33 lines of comments per 100 lines of source code. 0 marks for submissions with fewer or no comments
3. Performance Report: instruction count, register allocation, optimization notes.
4. Bonus (+5 marks): Use product of eigenvalues / singular values for determinant computation.

7. Evaluation Rubric (20 Marks)

Criterion	Marks	Description
Matrix-vector multiplication	3	Correct computation of Y
Determinant computation	5	By triangularization
Comparison flag logic	2	Sets correct flag result
RV32IMF instruction use	4	flw, fsw, fadd.s, fmul.s usage
Register allocation and loops	2	Efficient indexing
Performance report	4	Cycle analysis and optimization report, submit with source code
Bonus	+5	Determinant computation by eigenvalue/ singular value computation

8. Tools

Use RARS or Venus (VS Code) simulators supporting RV32IMF; Ripes optional for pipeline visualization.

9. Expected Output Example

Matrix-Vector Result Y: [91.0, 37.0, 31.0, 58.0, 25.0, 69.0]

Determinant Computation: 40.0

Comparison: $\Sigma Y = 311.0 \rightarrow \Sigma Y > \det(A) \rightarrow \text{comparison_flag} = 0$

10. Academic Integrity

All submissions must be individually authored. AI-generated or copied assembly will receive zero credit. 33 lines of comments per 100 lines of source code are mandatory for obtaining any marks (0 marks if not present).

Appendix A — Installing and Using RARS

RARS (RISC-V Assembler and Runtime Simulator) is a Java-based tool for running RISC-V programs.

Installation:

1. Download from <https://github.com/TheThirdOne/rars/releases>
2. Run: `java -jar rars1_6.jar`
3. Enable floating-point: Settings → Floating Point → Enabled (RV32F)

Usage:

- Assemble and run your program.
- Registers pane shows integer and float registers.
- Use syscalls: `li a7,2` to print float (fa0).

Appendix B — Using Venus with VS Code

Venus is a JavaScript-based RISC-V simulator for VS Code.

Setup:

1. Install VS Code → Extensions → 'Venus RISC-V Simulator'
2. Open your .S file → Run in Venus.
3. View output in terminal.

CLI Alternative:

```
npm install -g venus  
venus-run matvec_det_rv32imf.S
```

Venus supports most RV32IMF instructions and is ideal for development.

Appendix C — Test Matrix and Expected Output (Instructor Evaluation)

Matrix A (6×6):

```
[[1, 2, 3, 4, 5, 6],  
 [2, 1, 2, 1, 2, 1],  
 [3, 0, 1, 2, 1, 0],  
 [1, 2, 3, 4, 3, 2],  
 [2, 1, 0, 1, 2, 1],  
 [1, 3, 2, 4, 1, 5]]
```

Vector X: [1, 2, 3, 4, 5, 6]^T

Expected Results:

$Y = [91.0, 37.0, 31.0, 58.0, 25.0, 69.0]$

$\det(A) = 1 \times 1 \times 1 \times 4 \times 2 \times 5 = 40.0$

$\Sigma Y = 311.0 \rightarrow \Sigma Y > \det(A) \Rightarrow \text{comparison_flag} = 0$

Instructor Checkpoints:

- Correct Y vector (± 0.01 tolerance)
- Determinant = 40.0
- comparison_flag = 0